L2 Knowledge Graph — OpsMap

Implementation Summary · Dimension 2 of 3 (Process Mining)

Stack: FastAPI + pure-Python process mining over a captured workflow event log, with optional Apache AGE persistence · Frontend: Next.js + React Flow

1. What OpsMap is

Where **EntityMap** answers *"what exists and how is it related"*, **OpsMap** answers *"how does work actually flow"*. It is the process dimension of L2: a directly-follows graph (DFG) of the activities the crew-change process moves through and the transitions between them, **discovered from the events the system emits at runtime** — not drawn by hand. It mirrors the CognixOne / PM4Py process-mining model (DFG + variants + bottlenecks + conformance).

Unlike EntityMap (which requires AGE), OpsMap is mined in Python from an in-memory event log, so it works under the **fallback** backend too. AGE is only needed to *persist* the mined model alongside EntityMap as `(:Activity)-[:NEXT]->(:Activity)` edges.

2. Architecture & data flow

Every event WorkflowService relays through `_event_callback` is recorded into the OpsMap event log, keyed by `workflow_id` (the process-mining **case id**). Capture is wrapped in try/except so it can never break the live workflow or the WebSocket stream.

```
WorkflowService._event_callback(event_type, agent_name, data)
  ■■> ops_map.record_event(data['workflow_id'], event_type, agent_name, ts, data)
      ■■> in-memory event log  →  mining functions  →  REST API  →  React-Flow UI
```

Only meaningful, case-scoped events become activities; stream noise (agent_message, agent_thinking, model_usage, agent_tool_use, master_routing, master_waiting) is intentionally ignored. As of the latest change, a **curated slice of each event's data payload** is stored on the event too (crew names, compliance score, rejection/failure reason) so per-case detail views can show the actual record behind each step. The bulky compliance subgraph is dropped.

3. Activity vocabulary

Activity	Mined from event	Actor
Sign-Off Initiated	workflow_created	Master Agent
Crew Matching	agent_completed (Crew Matching Agent)	Crew Matching Agent
Travel Arranged	agent_completed (Travel Agent)	Travel Agent
Crew Notified	agent_completed (Notification Agent)	Notification Agent
Sign-Off Confirmed	crew_updated	Master Agent
Compliance Check	auto_compliance / sign_on_initiated	Master / Compliance Agent
Signed On (terminal · success)	crew_signed_on	Compliance Agent
Sign-On Rejected (terminal · rejected)	sign_on_rejected	Compliance Agent
Workflow Failed (terminal · failed)	workflow_failed	Master Agent

Crew Matching / Travel / Notification run **in parallel** in Phase 1, so their order varies between cases — that is real process behaviour and shows up honestly in the variants. Conformance treats them as an order-insensitive block.

4. What is mined

Process model (DFG) — `build_process_graph()`: nodes are activities (each carrying the number of distinct cases that hit it); edges are observed a→b transitions carrying **count** (frequency) and **avg_seconds**

(mean wait). Cycle time = last-minus-first event timestamp, averaged across cases.

Variants — `process_variants()`: cases bucketed by their exact ordered activity sequence, ranked by frequency, each tagged success / rejected / failed / in_progress.

Bottlenecks — `bottlenecks()`: DFG edges ranked by avg duration — where work waits longest (e.g. Sign-Off Confirmed → Compliance Check).

Conformance — `conformance()`: each case checked against the normative HAPPY_PATH with a partial-order rule (start at Sign-Off Initiated, contain every milestone, keep ordering with the 3 specialists free to interleave, end at Signed On). Non-conformant cases are returned with the reason they deviated.

Per-case records — `process_cases()`: each case with the actual data behind it — who signed off, who was signed on (or rejected/failed and why), compliance score, cycle time, and the ordered steps with their record-specific details.

5. REST API (/api/v1/graph/opsmap/...)

Endpoint	Returns
GET /opsmap/summary	cases, activities, transitions, variant count, conformance rate, avg cycle time
GET /opsmap/process	the React-Flow-ready DFG (nodes + edges with frequency/duration)
GET /opsmap/variants	distinct paths ranked by frequency, with outcome + cycle time
GET /opsmap/bottlenecks?limit=	slowest handoffs
GET /opsmap/conformance	happy-path conformance rate + per-case deviations
GET /opsmap/cases (new)	per-case records with crew identities + outcome reason + step details
POST /opsmap/persist	write the mined model into AGE (requires GRAPH_BACKEND=age)

All read endpoints work under both backends and return empty structures (not 503) before any workflow has run.

6. Frontend UI

The Graph page (`/graph`) has an EntityMap / OpsMap dimension toggle. The OpsMap view renders:

• **Process map** (React Flow) — activities laid out left→right by their place in the flow; the 3 parallel specialists share a column; terminals are colour-coded (green = Signed On, amber = Rejected, red = Failed); edges labelled "N× · duration"; the slowest handoff highlighted as the bottleneck.
• **Summary chips** — cases · activities · variants · conformance · avg cycle time.
• **Side panels** — Cases, Variants, Bottlenecks, Conformance.
• **Clickable nodes** — selecting an activity opens a detail panel: description, source event + actor, case/variant counts, bottleneck flag, transitions, and the **actual case records** that hit it (who signed off/on, score, failure reason).

Files: components/graph/OpsMapGraph.tsx, components/graph/OpsMapView.tsx, app/graph/page.tsx, lib/api.ts (opsMapApi).

7. Verified behaviour

With the 4 captured sample traces (2 happy, 1 rejection, 1 failure) via `seed_ops_map --demo`:

4	9	11	4	50%
cases	activities	transitions	variants	conformance

• Top bottleneck correctly identified as **Sign-Off Confirmed** → **Compliance Check** (the deliberately slow handoff in the rejection case).
• 4 variants surfaced with outcomes success, success, rejected, failed.
• The 2 happy-path cases conform regardless of specialist interleaving; rejection + failure do not.

8. How to run

```
# OpsMap self-populates as workflows run (events captured live).
# To preview before any live run, replay the captured sample traces:
cd backend
python -m L2Knowledge_graph.scripts.seed_ops_map --demo

# Persist the mined DFG into the AGE maritime graph (needs GRAPH_BACKEND=age):
GRAPH_BACKEND=age python -m L2Knowledge_graph.scripts.seed_ops_map --demo --persist
```

Note: the event log is in-memory per process. The seed script populates its own process; the live API process fills its log from real workflow events and the OpsMap endpoints read that live log.

9. Design alignment note

The implemented OpsMap is a **process-mining** dimension (Activity / NEXT, variants, bottlenecks, conformance). The baseline L2 design (L2_KNOWLEDGE_GRAPH_DESIGN.md §5.1) had specified a different concept under the same name — an **operational-state overlay** (Stage nodes with AT_STAGE / MATCHED_TO / BACKFILLS edges on existing Crew/Vessel nodes, plus a “who can backfill vessel V” query). These are not the same: the state overlay and its backfill query are **not yet implemented**. Reconciling the two (adopt process mining in the baseline doc, or also build the state overlay) is an open item.

Generated from the current implementation in backend/L2Knowledge_graph/ (ops_map.py, routes.py, services/workflow_service.py, frontend OpsMap components). Companion design: OPSMAP_DESIGN.md.